

L3 需求管理(2)

内容提要

- 估算用户故事
- 发布计划与迭代计划
- 把用户故事用于Scrum
 - 产品待办列表 (Product Backlog)
 - 迭代待办列表 (Sprint Backlog)
- 其他需求管理方法
- 案例分析

内容提要

- **估算用户故事**
- 发布计划与迭代计划
- 把用户故事用于Scrum
 - 产品待办列表 (Product Backlog)
 - 迭代待办列表 (Sprint Backlog)
- 其他需求管理方法
- 案例分析

估算用户故事

- 传统估算的问题
 - 选定生产率
 - 生产率是经验值
 - 如果一开始偏差大，也不能在项目过程中及时反馈
 - 估计与计划不能根据实际情况灵活调整
 - 瀑布模型难以准确反映项目进展
 - 后期发现偏差后难以进行适应性调整
- 思路
 - 较小粒度的需求较易估计，也更加灵活
 - 采用相对的估计
 - 采用基于本团队过去迭代的速度进行估计(“昨日天气”)

估算用户故事

- 故事点 (story point)
 - 团队可以自己定义合适的故事点
 - 理想天
 - 理想周
 - 一个标准工作量
 - 某种复杂度度量
 -
 - 团队共同估算达成一致
 - 不同人的估计可能不同
 - 采用扑克牌背靠背完成
 - 交流各自的考虑
 - 2-3轮完成统一



估算用户故事

- 故事点 (story point)
 - 相对估算
 - 三角测量：A与B、C相比，大还是小？



– 越大的估算精度越低

- 1,2,3,5,8,13,20,40...

估算用户故事

- 正确使用故事点的注意事项
 - 故事点是团队内的共识，**不同的团队可能标准不同**
 - 一个故事（比如史诗故事）分解成的小故事的故事点之和，**无需**与原故事的估算故事点相等
 - 故事继续分解成需要开发的任务时，任务的估算总和也不需要和故事的估算相等
 - 选择基准
 - 选择基准时可以选择最简单的故事，作为1，其他的参照1的几倍估算
 - 选择基准时选择中等的故事，作为3、4或5，其他的参照选定的版本进行估算

内容提要

- 估算用户故事
- **发布计划与迭代计划**
- 把用户故事用于Scrum
 - 产品待办列表 (Product Backlog)
 - 迭代待办列表 (Sprint Backlog)
- 其他需求管理方法
- 案例分析

发布计划

- 什么时候发布？
 - 初次发布的时间点，或者时间范围
 - 根据迭代次数估计
 - “一个带有主要可运行功能的初步版本”
 - “一个1.0版本”
- 发布中包含哪些功能？
 - 按照故事的优先级进行排列
 - “高” “中” “低” 优先级
 - 从系统功能发布的角度安排**故事优先级**

发布计划

- 优先级排序方法（MoSCow规则）
 - Must have 必须有
 - 系统的基本功能
 - Should have 应该有
 - 很重要，但短期内有替代解决方案
 - 如果时间允许，这类功能也是必需的
 - Could have 可以有
 - 如果没有时间，这类功能可以没有
 - Won' t have this time 这次不会有（但后续会）
 - 客户期望拥有，但同时承认在后续发布中再实现的功能

发布计划

- 排列故事的优先级
 - 由**客户**对故事的优先级进行排序
 - 考虑因素
 - 对**广泛**用户或客户的重要性
 - 对**少部分重要**用户或客户的重要性
 - 与其他故事的**关联性**
 - 先对故事进行估算，再确定优先级
 - 一个故事多久完成，会影响到优先级排定
 - 如果一个故事的不同细节有不同的优先级，那么拆分
 - 搜索工作→根据单个工作职位和单个工作地点搜索；根据多个职位、多个地点、多个薪酬范围进行复杂搜索

发布计划

- 高价值故事 vs 高风险故事
 - 风险驱动的螺旋模型着重早期消除风险(Boehm)
 - 而价值驱动则提倡先做 “the juicy bits” (“油水最多的部分”)(Gilb)
- 敏捷：先做最有价值的
- 如何考虑风险？
 - 让客户了解
 - 由客户决定
 - 技术团队与客户紧密合作



Barry Boehm



Tom Gilb

发布计划

- **高风险故事** 往往与基础性或非功能性需求有关
 - 性能需求——往往与架构设计、新的算法研究有关
 - 但需求往往被排到较低的优先级（因为没有也能用）
 - 客户期望将来总能进行重构系统或进行扩展优化
 - 技术团队需要客观说明延迟这些需求的实现可能带来的架构风险或者额外增加的重构代价
 - 直接的架构期望
 - 客户与技术团队已商讨确定某种技术架构（比如三层架构）
 - 但客户并未写下相关故事
 - 技术团队实现时，可能先用二层架构完成主要功能
 - 可能需要加入“用户安装时能分别安装客户端、中间层和服务器后端”这样的故事

发布计划

- 根据故事点预计工期
 - 利用“速率” (velocity)：每轮迭代完成多少个故事点
 - 如何获取“速率”？
 - 使用历史值
 - 最好的选择，但是.....
 - 同样的团队做相似的项目极为罕见
 - 执行一轮迭代后，用这一轮迭代的速率
 - 但并不是总是可行
 - 猜测
 - 基于故事点与“理想日”的对应关系
 - 实际开发人天折算成1/3或者1/2的“理想日”或故事点
 - » 实际可用的时间需要排除许多“杂事”

发布计划

不要迷信发布计划！

迭代计划

- 讨论故事
- 分解任务：从故事中分解出更易于执行的任务
- 承担职责：开发人员主动认领各个任务
- 估算并确认：开发人员单独估计他们承担的任务

通过迭代计划会议
获取有关用户故事的更多细节

迭代计划

- 讨论故事
 - 通过所有人员（开发、测试、客户）共同参加的讨论对故事的细节进行展开
 - 细节展开的目的是让开发人员可以分解出任务
 - 没必要过分深入所有细节
 - 抓住故事的目标
 - 关键细节仍然需要开发人员在开发过程中与客户保持有效的沟通

迭代计划

- 分解任务
 - 故事本身已经是“小的”了
 - 如果单人工作，也可以直接作为任务
 - 有些故事仍然需要分解为更小的任务，以便项目执行
 - 需要多人合作开发
 - 比如前端和后端、整体逻辑与某个特定步骤所用的技术
 - 防止一些细节被遗忘
 - 任务分解是团队共同的工作，每人都积极参与其中
 - 比如除了功能外还需要一个手册
 - 任务没有强制的大小（比如没有必须要在3-5小时内完成这种要求）
 - 但应当能够帮助估算或者开启多人合作

迭代计划

- 分解任务——例子

求职者可以通过地区、薪水范围、职位、公司名称和发布日期之类的属性来搜索工作



怎么搜？

前后端

备忘

- 编写根据地区、薪水范围等属性搜索的基本界面
- 编写多地、多职位搜索的高级搜索界面
- 确定前后端消息交换格式
- 后台查询服务编写
- 对查询功能写下相应的用户手册文档

迭代计划

- 承担职责
 - 开发人员认领任务
 - 任务由单人负责到底，虽然他可以去找“帮手”
 - 任务最终是**团队中每个人的**
 - 任何人有困难，其他人应当尽量予以帮助
 - 必要时任务认领和承诺可以做出调整
 - 比如拆分或者换人负责

迭代结束时，**不应该**有人说：
“我完成了我的任务，他还有一个任务没完成”

迭代计划

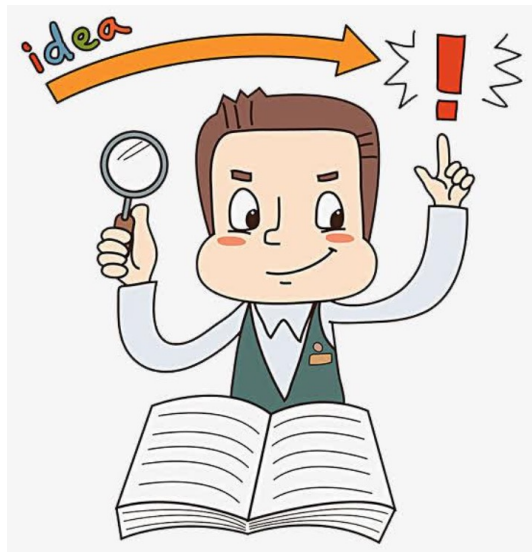
- 估算并确认
 - 每人估算（例如按照理想日计算故事点）
 - 任务的分解估算不需要与故事的总点数相等

任务	责任人	估算（故事点）
编写根据地区、薪水范围等属性搜索的基本界面	张三	2
编写多地、多职位搜索的高级搜索界面	张三	2
确定前后端消息交换格式并集成	王五	1
后台查询服务编写	王五	3
对查询功能写下相应的用户手册文档	张三	2

- 每人看看是否能在迭代中完成，尽量有把握，并做出承诺

迭代计划

- 如果客户有新的想法.....
 - 避免在迭代中临时增加故事或者修改已经讨论好的故事细节
 - 可以把需要的功能写成新的故事，放到后续的迭代中



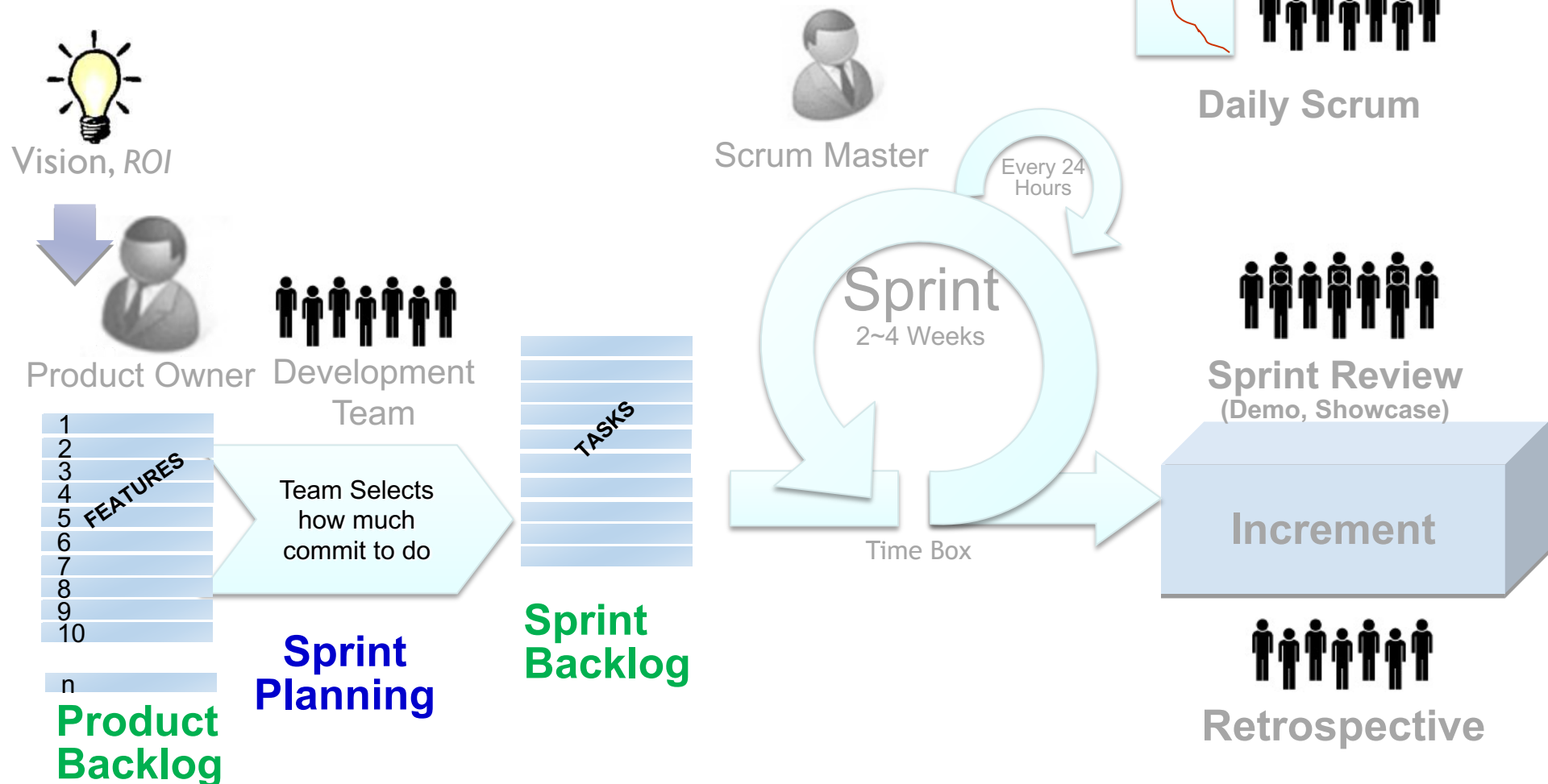
- 一个**完整的功能**才是有价值的
- 让客户**看到**那个已经完成的功能
- 然后根据**成本和价值**来决定是否要去实现那个新的idea

内容提要

- 估算用户故事
- 发布计划与迭代计划
- **把用户故事用于Scrum**
 - 产品待办列表 (Product Backlog)
 - 迭代待办列表 (Sprint Backlog)
- 其他需求管理方法
- 案例分析

Scrum与用户故事

- 产品待办列表Product Backlog
- 迭代待办列表Sprint Backlog



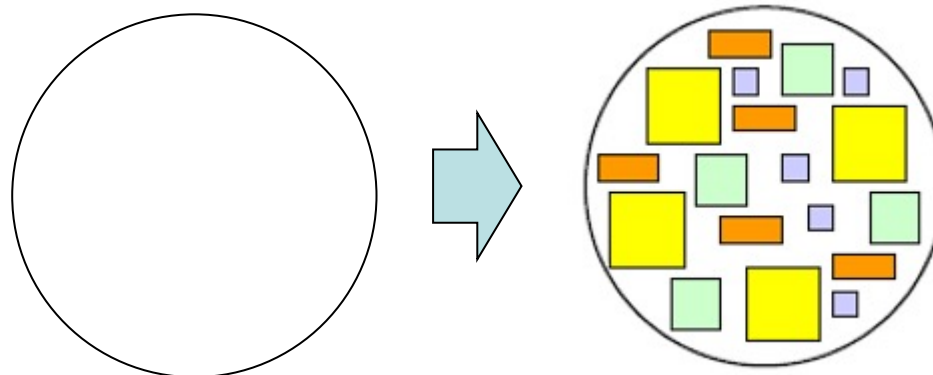
Scrum与用户故事

- 产品待办列表 (Product Backlog)
 - 当**不采用**用户故事时，产品待办列表可能包括多种工作任务
 - 新功能；要调研的问题；要解决的缺陷.....

编号	任务
1	编写工作查询功能
2	编写工作详情查看功能
3	重构Login类，使之能抛出登录异常
4	编写工作信息发布功能
5	重构Authentication类，使之支持多种登录方式扩展
6	编写第三方验证登录功能
...	

Scrum与用户故事

- 产品待办列表 (Product Backlog)
 - 采用用户故事时，Product Backlog中的所有条目都是故事
 - 容易进行排序
 - 容易做出取舍
 - 一开始不需要维护非常细节的故事列表，但尽可能由所有人一起完成最初需求的收集
 - 确定角色，根据角色来书写故事



Scrum与用户故事

- 迭代任务列表 (Sprint Backlog)
 - Sprint计划会议
 - 整天的会议，确定当前迭代要完成的目标
 - 从Product Backlog中**选取高优先级的故事**填充到当前迭代中
 - 根据估算确定取多少故事
 - **细化讨论**每个故事
 - 拆分具体任务
 - 如果Product Backlog中混有技术类任务，那么要说明该任务对客户价值
 - 形成**Sprint Backlog**
 - 小的故事或者由故事细化出的任务，然后由开发者认领

Scrum与用户故事

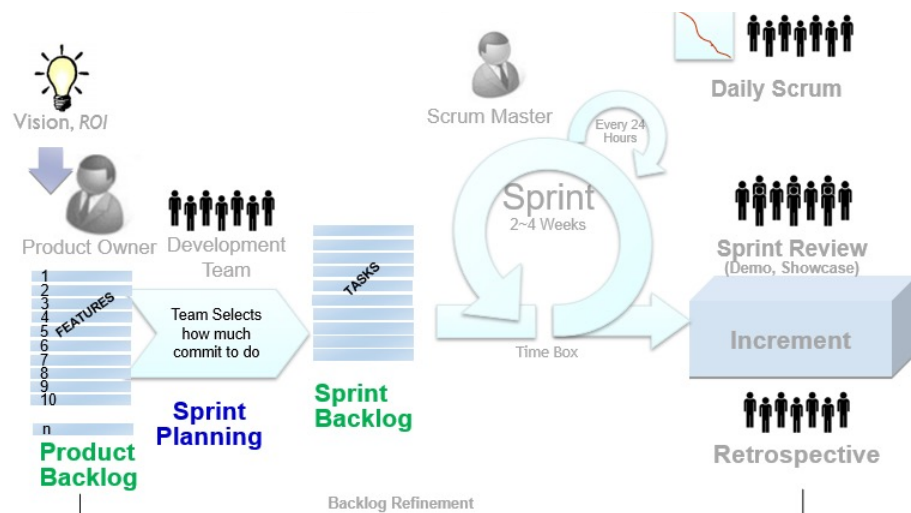
- 迭代任务列表（Sprint Backlog）
 - 每日Scrum站会
 - 每天固定时段的简短会议（通常不超过15分钟）
 - 根据Sprint Backlog中的任务，由Scrum Master提问：
 - 昨天做了什么？
 - 今天打算做什么？
 - 目前有什么困难？
 - 不是开发人员向Scrum Master的工作汇报，而是在团队内部成员之间的承诺
 - 不是具体的设计讨论、不是解决问题
 - 不询问实现细节，不讨论“何时完成”
 - 估算在白板中各自更新

Scrum与用户故事

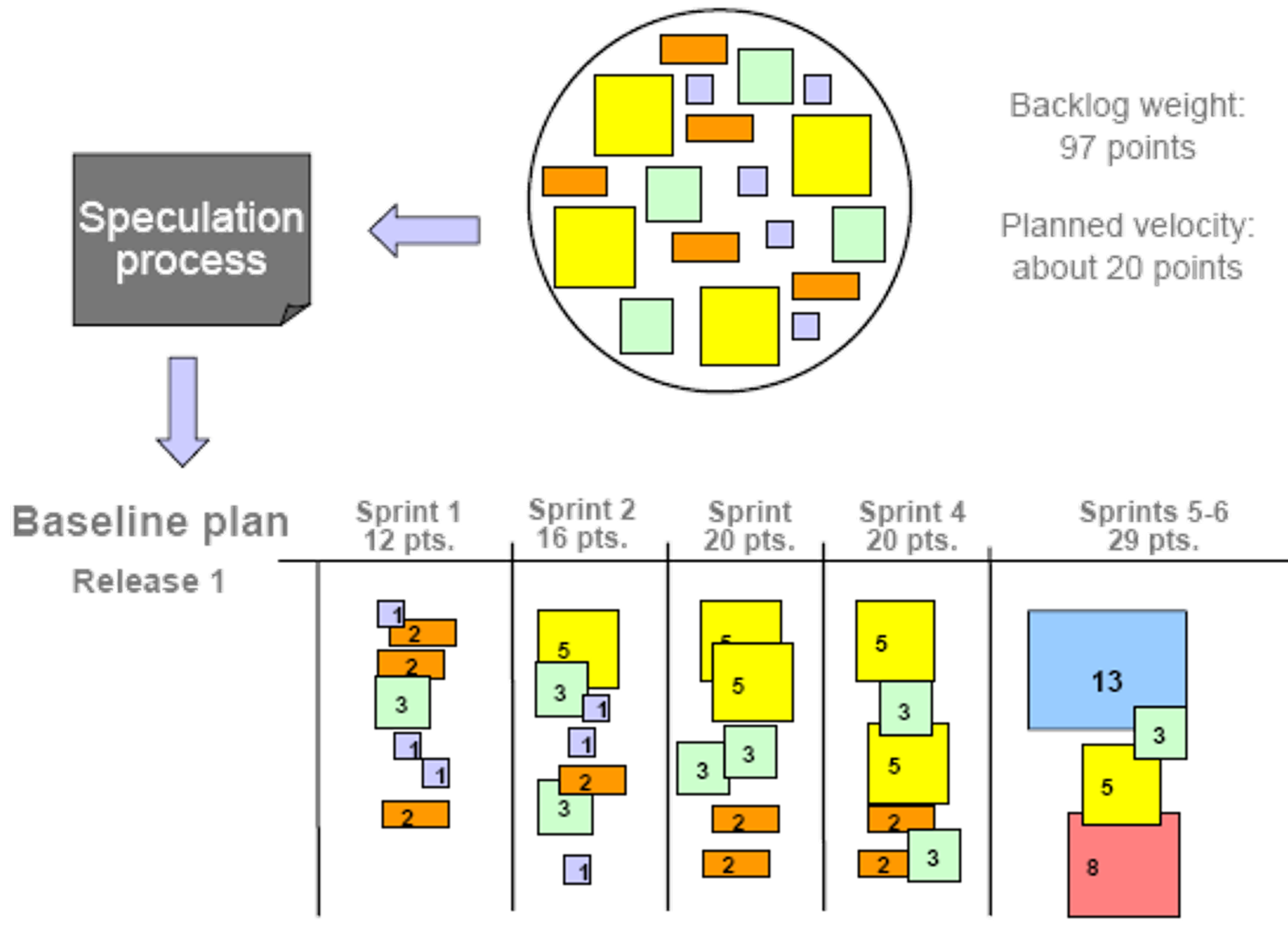
- 迭代任务列表 (Sprint Backlog)
 - Sprint评审会议 (Review Meeting)
 - 每个Sprint发布一个**增量** (以前称作 “潜在可交付产品增量” PSPI)
 - 先不管运维和客服，从产品角度完成功能增量
 - 在评审会议中，利用客户可感知的价值，来**演示增量**
 - 关注目标，而非具体一个个的任务
 - 故事具有可演示的效果
 - **演示新功能**，而不是做成PPT
 - 保持非正式会议的形式
 - 关注产品本身
 - 应该是Sprint的自然而然的结果

Scrum与用户故事——小结

- 从Product Backlog中选取故事
 - 建立迭代目标
- 每次迭代（Sprint）开始时
 - 讨论故事，并细化成任务
 - 建立Sprint Backlog
 - 领取任务（并估算）
- 通过每日站会沟通进度
 - 但不解决具体问题
- 完成迭代时
 - 根据故事进行演示



Scrum与用户故事



内容提要

- 估算用户故事
- 发布计划与迭代计划
- 把用户故事用于Scrum
 - 产品待办列表 (Product Backlog)
 - 迭代待办列表 (Sprint Backlog)
- **其他需求管理方法**
- 案例分析

其他需求管理方法

- 用户故事地图
- 用户故事树
- 依赖关系图

其他需求管理方法

- 用户故事地图

史诗故事（按主要功能或业务流程）

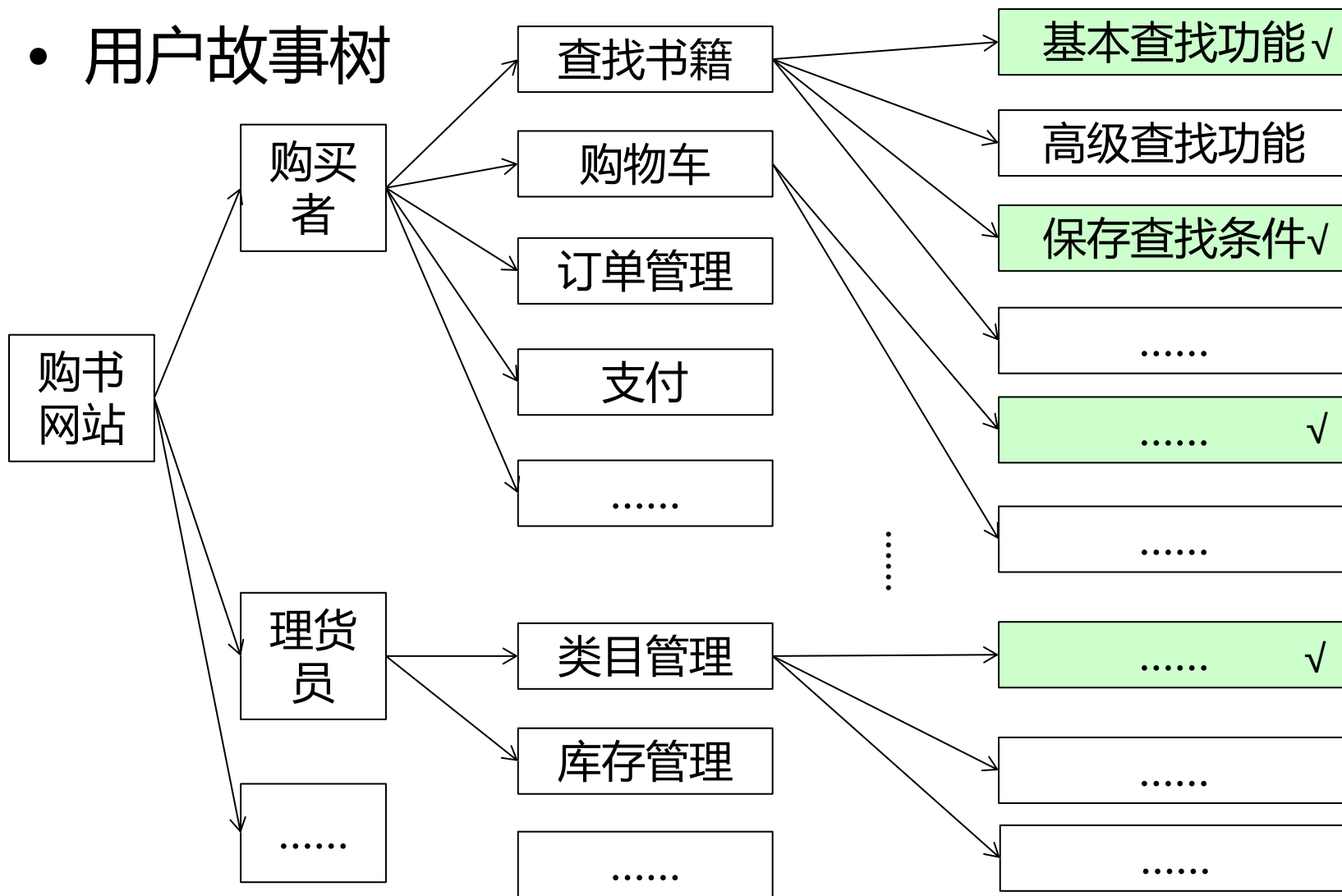


每个史诗故事按优先级拆分为小的故事

根据交付目标选择一组故事进行迭代开发

其他需求管理方法

- 用户故事树



产品

用户

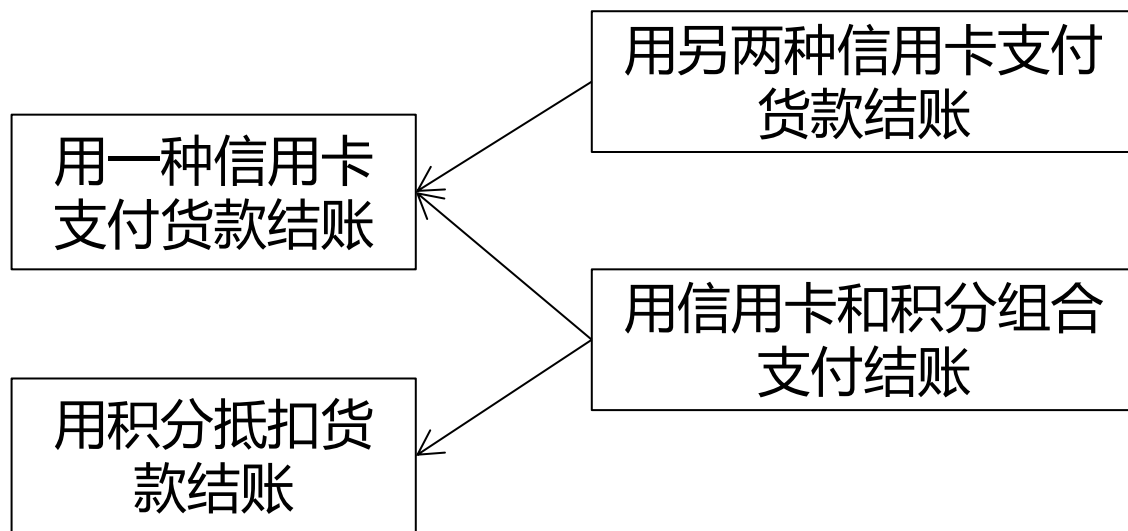
特性

用户故事

其他需求管理方法

- 依赖关系图

- 虽然我们希望故事之间是相互独立的，但现实中依赖往往不可避免



内容提要

- 估算用户故事
- 发布计划与迭代计划
- 把用户故事用于Scrum
 - 产品待办列表 (Product Backlog)
 - 迭代待办列表 (Sprint Backlog)
- 其他需求管理方法
- 案例分析

课程项目实践

- 实践要求

- 对发布目标进行初步计划
- 讨论制定Product Backlog
- 编写相应的Epic以及细化故事
- 讨论迭代，细化任务
- 体会对故事点的认知以及估算效果（比如实际可用时间会打多少折扣）
- 体会每次迭代的固定time box
 - 考虑6月10日或17日为阶段性版本发布日
- 采用代码扫描、自动化测试、代码评审、持续集成和持续交付等技术提升开发质量和效率

Next Lecture

- 质量管理(1)